

Informatik 2

Kapitel 7 – Eigene Datenklassen implementieren

Inhaltsverzeichnis

7.1 Einleitung	2
7.2 Fallstudie: Grafischer Editor	3
7.3 Fallstudie: Angestelltenverwaltung	5
7.4 Allgemeine Regeln der Implementierung	13
7.4.1 Aufbau einer Klasse	13
7.4.2 Attribute einer Klasse	13
7.4.3 Konstruktoren einer Klasse	14
7.4.4 Verwaltung von Attributen	15
7.4.5 equals überschreiben	17
7.4.6 hashCode überschreiben	19
7.4.7 Containerklassen zu Datenklassen	19
7.5 Einführung in Javadoc	19
7.6 Annotationen	22

7.1 Einleitung

An dieser Stelle haben wir die Grundlagen in der objektorientierten Programmierung in Java und der Benutzung der Java-API abgeschlossen. Ab jetzt wollen wir **systematisch in die Entwicklung einer größeren komplexen Anwendung** in Java einsteigen. Eine solche Anwendung besteht grundsätzlich aus den folgenden Komponenten:

- **Eigene Datenklassen**

Diese dienen zur Repräsentation und Verwaltung der Daten der Anwendung. Solche Klassen nennen wir ab jetzt **Fachkonzeptklassen**. Hierfür haben wir schon einige Beispiele kennengelernt, wie die Klassen [Circle](#), [Rectangle](#), [Form](#) und [FormContainer](#). Die systematische und vollständige Entwicklung solcher Klassen in Java besprechen wir in diesem Kapitel. Dazu gehören auch eine gute **Dokumentation** der Klassen inklusive sogenannter **Annotationen** – dazu gibt es Unterkapitel am Ende dieses Kapitels mit einem kurzen Überblick zum Nachschlagen. Den detaillierten Entwurf solcher Klassen in UML diskutieren wir in Kapitel 9.

- **Eigene (geprüfte oder ungeprüfte) Ausnahmeklassen**

Diese gehören zu den Datenklassen und repräsentieren Fehler, die bei der Verwaltung der Daten auftreten können. Diese gehören auch zu den Fachkonzeptklassen und verbessern die Wartbarkeit und Bedienung des Programms, indem sie nützliche Informationen an den Entwickler und den Benutzer ausgeben. Diese Klassen haben wir bereits im letzten Kapitel diskutiert.

- **Eigene Fenster-Klassen**

Diese dienen als grafische Benutzerschnittstelle zur Bedienung der Anwendung über grafische Darstellungen, Eingabe von Daten in Textfelder und Benutzeraktionen. Wir betrachten diese in Kapitel 8.

- **Eigene Datenspeicherungsklassen**

Diese stellen Methoden zum persistenten Speichern und Laden bereit, zum Beispiel mit Datenbanken oder Textdateien. Diese werden wir in den Kapiteln 13 behandeln.

Zudem lernen wir in den Kapiteln 9 - 12 Möglichkeiten kennen, uns aus verschiedenen Perspektiven einen Überblick über alle diese Komponenten einer Anwendung und ihr Zusammenspiel zu verschaffen.

Dieses Kapitel veranschaulicht die Entwicklung eigener Fachkonzeptklassen anhand von zwei Beispielanwendungen: Einem grafischen Editor zum Zeichnen von geometrischen Formen wie Kreisen und Rechtecken und ein Programm zur Verwaltung von Angestellten in einer Firma. Um diese beiden größeren Programme zu erstellen, müssen wir natürlich erst einmal wissen, was unsere Anforderungen an die beiden Programme sind. Im Berufsleben werden Sie dafür normalerweise ein sogenanntes **Pflichtenheft** erhalten – dieses enthält möglichst detaillierte Beschreibungen des Auftraggebers, wie sich das Programm, das Sie entwickeln sollen, verhalten und wie es aussehen soll. Da ein solches Dokument meist viele Seiten enthält, werden wir ein solches hier nicht zeigen, sondern unsere Anforderungen in aller Kürze zusammenfassen.

Für die Verwaltung der Daten des grafischen Editors haben wir in den bisherigen Kapiteln bereits eine solche kurze Beschreibung eingeführt (Kapitel 2) und daraus Fachkonzeptklassen ent-

wickelt (Kapitel 2 - 6). Das Programm zur Angestelltenverwaltung ist ein komplett neues Beispiel.

7.2 Fallstudie: Grafischer Editor

Die Fachkonzeptklassen zum grafischen Editor haben wir bereits in den Kapiteln 2 - 6 vollständig entwickelt. Dabei haben wir allerdings in den jeweiligen Kapiteln unterschiedliche Schwerpunkte gelegt. Deshalb fassen wir hier die bisherige UML-Modellierung und Java-Implementierung noch einmal übersichtlich zusammen.

Dazu zuerst noch einmal die Beschreibung der Anwendung:

Beispiel 7.1: Grafischer Editor

Es soll ein grafischer Editor zum Zeichnen von **Kreisen** und **Rechtecken** realisiert werden, der später bei Bedarf leicht um weitere geometrische Formen erweitert werden kann. Ein Kreis hat eine *Position*, einen *Radius* und eine *Randfarbe*. Rechtecke werden durch ihre *Länge*, *Breite*, *Position* und *Randfarbe* beschrieben. Positionen sind **Punkte** mit *nicht-negativen ganzzahligen Koordinaten* auf der 2-dimensionalen Zeichenfläche und repräsentieren jeweils den Mittelpunkt der geometrischen Formen. Der Benutzer soll per Mausklick Kreise und Rechtecke *fester Größe zeichnen* können. Zudem soll er Kreise und Rechtecke mit gedrückter Maustaste *aufziehen* können. Alle geometrischen Form sollen mit der Maus *ausgewählt* werden können, indem der Benutzer mit der Maus innerhalb der Form auf die Zeichenfläche klickt. *Ausgewählte* Formen sollen durch eine *andere Randfarbe* dargestellt werden. Außerdem soll ebenfalls durch eine *andere Randfarbe* dargestellt werden, wenn sich die Mausposition innerhalb einer Form befindet. *Ausgewählte* Formen sollen *mit der Maus und mit den Pfeiltasten verschoben* und durch Schaltflächen-Klicks *kopiert oder gelöscht* werden können.

In Abbildung 1 sieht man einen Gesamtüberblick über die Datenklassen des grafischen Editors in UML. Hierbei haben wir die gezeigten Datenklassen jeweils bereits in folgenden Kapiteln besprochen:

- Point: Kapitel 2 (ohne Check-Methoden und toString-Methode, ohne Ausnahmebehandlung) und Kapitel 6.
- Circle: Kapitel 2 (ohne Check-Methoden und toString-Methode, ohne Ausnahmebehandlung, ohne Oberklasse), Kapitel 4 und Kapitel 6.
- Rectangle: Kapitel 4 (ohne Check-Methoden, ohne Ausnahmebehandlung) und Kapitel 6.
- Form: Kapitel 4 (ohne Check-Methoden, ohne Ausnahmebehandlung) und Kapitel 6.
- FormContainer: Kapitel 5.
- IllegalDimensionException: Kapitel 6.

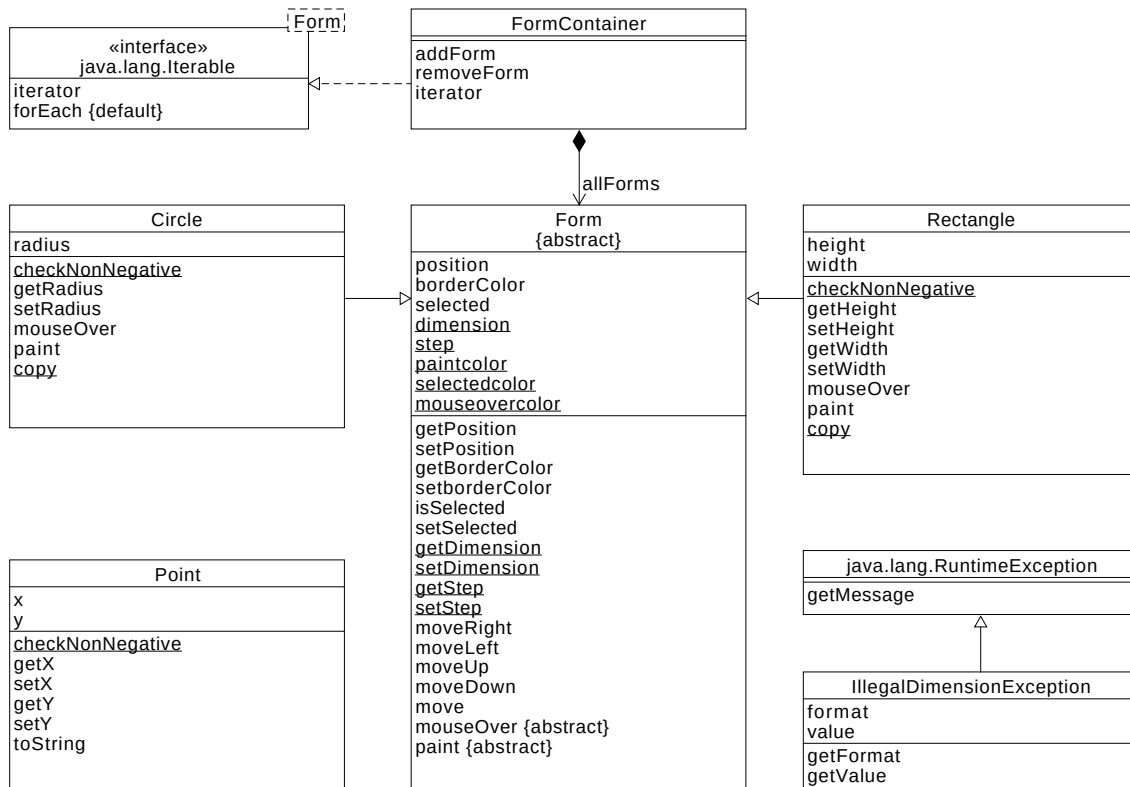


Abbildung 1: UML-Überblick über den grafischen Editor (Datenklassen).

Beachte dabei die folgenden kleinen Änderungen zu bisher gezeigten Versionen von Teilen des Modells:

- Keine der Formen überschreibt `toString`, da wir uns auf die rein grafische Ausgabe der Formen beschränken.
- Das Modell enthält keine Klasse `NumberCircle`, da diese Form nicht Teil der Anwendungsbeschreibung ist. Diese Klasse diente in Kapitel 4 nur der zusätzlichen Illustration von Vererbung.
- Die Klasse `Form` implementiert nicht die Schnittstelle `Comparable`, da eine solche Funktionalität nicht der Anwendungsbeschreibung entnommen werden kann. Diese Implementierung diente in Kapitel 4 nur der Illustration von Schnittstellen.
- Das Modell zeigt nicht, dass alle Klassen von `java.lang.Object` erben, da dies keine Designentscheidung ist, sondern immer automatisch der Fall ist.

Die Überprüfung, ob der an eine der Set-Methoden für den Radius (Klasse `Circle`) sowie für Länge und Breite (Klasse `Rectangle`) übergebene Wert ein ungültiger negativer Wert ist, lagern wir (wie in der Klasse `Point`) in eine separate Check-Methode `checkNonNegative` aus. Hier bietet der aktuelle Entwurf sicher noch Verbesserungsmöglichkeiten, denn wir implementieren in drei verschiedenen Klassen dieselbe Methode. Eventuell ließe sich diese in eine gemeinsame Schnittstelle

oder in eine gemeinsam benutzte Utility-Klasse auslagern.

Ansonsten berücksichtigt das Modell schon einige Prinzipien guten (objektorientierten) Designs. So lässt sich das Modell zum Beispiel leicht durch weitere Unterklassen von `Form` um zusätzliche geometrische Formen erweitern. Dabei muss keinerlei Änderung in bereits bestehenden Klassen vorgenommen werden. Dies war auch bei der Ergänzung des Modells um die Klasse `FormContainer` der Fall. Das ist ein wichtiges Prinzip des modularen Aufbaus von Softwaresystemen:

Eine Anwendung sollte um zusätzliche Software-Module erweitert werden können, ohne bestehende Module ändern zu müssen.

Das erreicht man im Falle des Containers durch eine **lose Kopplung** der Module. In Fall der Anbindung des Containers hat zwar der Container Zugriff auf andere Klassen, nämlich die Klasse `Form`:

Der Container kennt die verwalteten Formen.

Aber keine der anderen Klassen hat Zugriff auf den Container:

Formen kennen nicht den Container.

Das wird im UML-Diagramm durch die Pfeilrichtung der **besteht aus-Beziehung** zwischen dem Container-Objekt und den verwalteten Form-Objekten vorgegeben (beziehungsweise ausgedrückt).

Auch große Teile der Java-Implementierung der einzelnen Klassen wurde bereits in der vorherigen Kapiteln gezeigt, wobei einzelne Methoden oder Konstruktoren zur eigenen Übung überlassen wurden.

Übung: Spätestens jetzt sollten Sie zur Übung das gezeigte Modell komplett mit allen Details implementieren, bereits vorhandenen Code nachvollziehen und verstehen, und dann Ihre Implementierung exemplarisch mit einem kleinen Hauptprogramm testen.

7.3 Fallstudie: Angestelltenverwaltung

Als zweite Fallstudie betrachten wir nun die angesprochenen Angestelltenverwaltung. Diese wollen wir nach folgender Beschreibung entwerfen und implementieren:

Beispiel 7.2: Systembeschreibung einer Angestelltenverwaltung

Es soll ein System zur Verwaltung der **Angestellten** einer Firma realisiert werden. Für jeden Angestellten sollen dessen *ID*, *Name* und der *Vertragsbeginn* gespeichert werden. Die *ID* soll dabei eindeutig sein und fortlaufend vergeben werden. Der *Name* soll mit einem Großbuchstaben beginnen und mindestens die Länge 2 haben. Der *Vertragsbeginn* darf nicht vor dem 1.1.2000 liegen. Zudem kann jeder Angestellte einen anderen Angestellten als *Vorgesetzten* haben. Die Beziehung zwischen Angestellten und ihren Vorgesetzten darf dabei selbstverständlich keine Zyklen bilden. Manche Angestellte sind **Zeitangestellte**. Im Vergleich zu anderen Angestellten *endet bei diesen der*

Vertrag zu einem festgelegten Zeitpunkt, es ist also zusätzlich ein Vertragsende festgelegt. Dieses darf natürlich nicht vor dem Vertragsbeginn liegen.

Abbildung 2 zeigt eine aus der Beschreibung abgeleitete Übersicht über die Datenklassen der Angestelltenverwaltung in UML.

Aus der Beschreibung leiten wir zuerst die Klasse Employee ab mit den Objektattributen id, name und beginOfContract. Vielleicht stellen wir uns dann die Frage, ob wir Zeitangestellte nicht durch ein zusätzliches Attribut isTemporaryEmployee in der Klasse Employee kenntlich machen können. In diesem Fall müssten wir dann aber auch das Attribut endOfContract in die Klasse Employee aufnehmen. Dieses Attribut hätte dann für alle Nicht-Zeitangestellten keinen Wert. Trotzdem würde im System später aber Speicherplatz dafür vorgesehen – wird würden also Speicherplatz verschwenden. Zudem wäre die Klasse Employee unnötig lang und würde speziellen Code nur zur Verwaltung von Zeitangestellten enthalten.

Die bessere Lösung ist, eine Unterklasse TemporaryEmployee von Employee zu bilden. Diese erweitert Employee dann um das Attribut endOfContract (und dessen Verwaltung). Denn es gilt: Jeder Zeitangestellter **ist ein** Angestellter, nur mit zusätzlichen Eigenschaften. Durch die Vererbungsbeziehung erbt TemporaryEmployee alle Attribute und Methoden von Employee. Wir erzeugen bei der Implementierung also keinen redundanten Code.

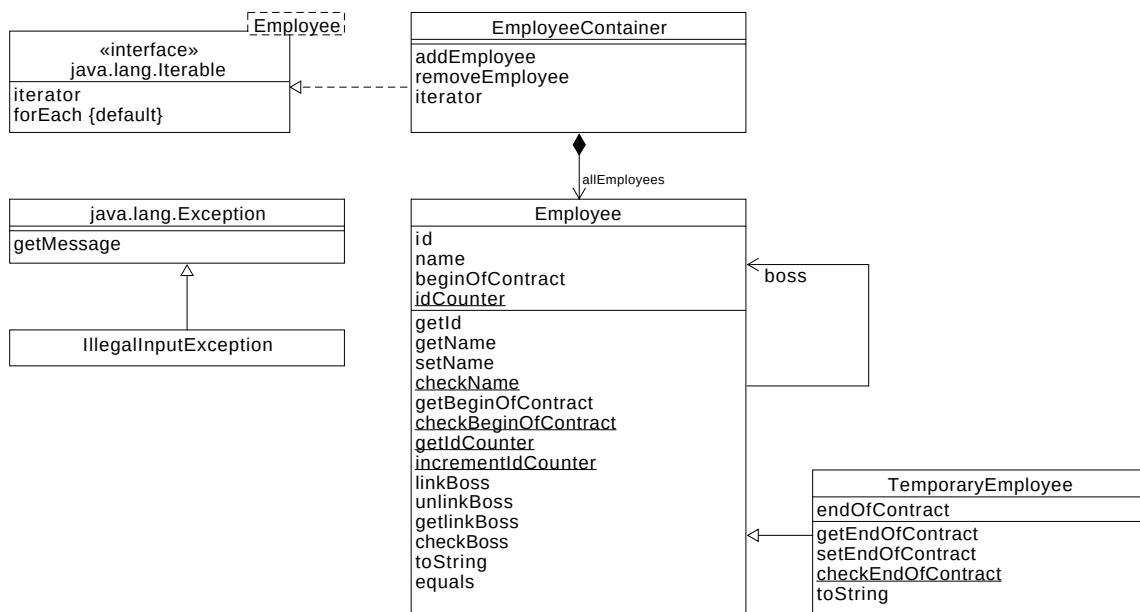


Abbildung 2: Übersicht über die Datenklassen der Angestelltenverwaltung in UML.

Dadurch wird die Klasse Employee zur Oberklasse von TemporaryEmployee. Da wir Objekte vom Typ Employee anlegen wollen (denn es gibt ja auch unbefristete Angestellte), wir diese Oberklasse **nicht abstrakt**.

Die Modellierung enthält folgende Besonderheiten, die einer Erklärung bedürfen:

1. Ungültige Attributwerte

Die Beschreibung enthält mehrere Hinweise auf ungültige Attributwerte. Zum Beispiel müssen Namen mindestens die Länge 2 haben. Für jede solche Datenstrukturinvariante sehen wir eine Checkmethode vor. Für ungültige Attributwerte verwenden wir diesmal eine geprüfte Ausnahme (`IllegalArgumentException`). Im Gegensatz zum grafischen Editor aus der vorherigen Fallstudie kann ein Benutzer hier eine Eingabe tätigen, die einer Datenstrukturinvariante widerspricht. Die Eingabe ungültiger Werte (zum Beispiel durch Tippfehler) kann nicht verhindert werden und sollte daher eine Fehlerbehandlung nach sich ziehen. Diese kann etwa so aussehen, dass der Benutzer durch einen Warndialog darauf hingewiesen wird, dass der Name nicht korrekt ist und er einen neuen eintippen soll.

2. Überschreiben von `toString`

Da die Ausgabe diesmal keine Zeichnung ist (wir beim Editor), überschreiben wir in allen Datenklassen `toString`. Die von `toString` gelieferte Zeichenkette benutzen wir für die Ausgabe von Angestellendaten in der grafischen Benutzeroberfläche (zum Beispiel in Textfeldern).

3. Fehlende Set-Methoden

Die ID `id` soll eine Konstante sein, die nach dem erstmaligen Setzen nicht mehr verändert werden kann. Deswegen sehen wir hier keine Set-Operation vor. Der erstmalige Wert wird später direkt im Konstruktor gesetzt. Das gleiche gilt für den Vertragsbeginn (`beginOfContract`). Möchte man auf Set-Methoden nicht verzichten, dürfen diese später nur intern verwendbar sein und müssen deshalb die Sichtbarkeit `private` bekommen.

4. Verwaltung der ID

Die IDs von neuen Angestellten sollen laut Beschreibung fortlaufend vergeben werden. Dazu legen wir ein Klassenattribut `idCounter` an, das immer den Wert der nächsten zu vergebenden Angestellten-ID enthält. Wird ein neuer Angestellter angelegt, wird dieses mit der Methode `incrementIdCounter` hochgezählt. Wie üblich sehen wir zudem eine Get-Methode vor.

5. Darstellung der Vorgesetzten-Beziehung

Jeder Angestellte kann laut Beschreibung einen Vorgesetzten haben. Dabei handelt es sich um eine Beziehung zwischen zwei `Employee`-Objekten. Solche Objektbeziehungen werden in UML über Pfeile zwischen Klassen (oder Schnittstellen) dargestellt. Eine erste solche Beziehung haben wir in der letzten Fallstudie zwischen `FormContainer` und `Form` kennengelernt. Dort handelte es sich um eine spezielle Beziehung – eine *besteht aus*-Beziehung – die wir durch einen speziellen Pfeil mit einer Raute modelliert haben. Die Vorgesetzten-Beziehung ist keine spezielle Beziehung und wird deshalb durch einen normalen Pfeil angegeben. Dieser ist am Pfeilende mit einer sogenannten *Rolle*, der Rolle `boss`, beschriftet, denn einer der Angestellten hat in der Beziehung genau diese Rolle. Die Beziehung wird wie folgt in Pfeilrichtung gelesen: *Employee hat boss*. In Java übernehmen wir nachher die Rolle `boss` als Referenz-Attribut in die Klasse `Employee`. In UML verwenden wir zwingend die Pfeilnotation, da diese Darstellung übersichtlicher ist.

6. Verwaltung der Vorgesetzten-Beziehung

Die Verwaltungsoperationen für die Rolle `boss` sind

- `linkBoss`:
Legt den Vorgesetzten eines Angestellten fest (dadurch wird *die Beziehung aufgebaut*).
- `unlinkBoss`:
Löscht den Vorgesetzten eines Angestellten (dadurch wird *die Beziehung abgebaut*).
- `getlinkBoss`:
Frägt den Vorgesetzten eines Angestellten ab (dadurch wird **die Beziehung gelesen**).
- `checkBoss`:
Überprüft, dass die Vorgesetzten-Beziehung wie gefordert nicht zyklisch wird. Denn natürlich darf es beispielsweise nicht vorkommen, dass Angestellter a Vorgesetzter von b ist, b wiederum Vorgesetzter von c ist, und schließlich c Vorgesetzter von a ist. Die Methode `checkBoss` ist im Gegensatz zu den anderen und bisherigen Check-Methoden keine Klassen-, sondern eine Objektmethode, da sie auf ein `Employee`-Objekt angewendet wird, für das ein neuer Vorgesetzter eingetragen werden soll.

Wie man sieht, gibt es für die Methoden zur Verwaltung von Beziehungen andere Namenskonventionen als bei den bisherigen Methoden zur Verwaltung von Attributen.

7. Überschreiben von `equals`

Schließlich wollen wir später in Java die `Object`-Methode `equals` geeignet überschreiben, um festzulegen, wann wir zwei verschiedene `Employee`-Objekte als gleich ansehen wollen. Das wird genau dann der Fall sein, *wenn sie die gleiche ID haben*. Denn so können wir alle `Employee`-Objekte in einer Sammlung verwalten und mit `contains` überprüfen, dass es keine zwei Angestellten mit derselben ID gibt (vergleiche die Implementierung von `contains` in Java).

Wie in der letzten Fallstudie verwalten wir alle zur Laufzeit erstellten Objekte in einer eigens angelegten Containerklasse.

Wir beginnen mit der Implementierung der Klasse `Employee`:

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.Optional;

public class Employee {

    private final int id;
    private final LocalDate beginOfContract;
    private String name;
    private static int idCounter = 1000;
    private Employee boss;

    public Employee(String name, LocalDate beginOfContract) throws
        IllegalArgumentException {
        setName(name);
        if (!checkBeginOfContract(beginOfContract)) {
            throw new IllegalArgumentException("begin of contract not
                valid");
        }
        this.beginOfContract = beginOfContract;
        this.id = idCounter;
    }
}
```



```

        boss = null;
        incrementIdCounter();
    }

    // Methoden für name, beginOfContract, id, idCounter: Übung

    public void linkBoss(Employee boss) throws IllegalArgumentException
    {
        if (!this.checkBoss(boss))
            throw new IllegalArgumentException("Relationship to boss
            must be acyclic");
        this.boss = boss;
    }

    public void unlinkBoss() {
        this.boss = null;
    }

    public Optional<Employee> getlinkBoss() {
        return Optional.ofNullable(this.boss);
    }

    private boolean checkBoss(Employee newBoss) {
        if (newBoss.equals(this)) {
            return false;
        } else if (newBoss.boss == null) {
            return true;
        } else {
            return this.checkBoss(newBoss.boss);
        }
    }

    @Override
    public boolean equals(Object o) {
        if (o == null)
            return false;
        if (!(o instanceof Employee))
            return false;
        Employee a = (Employee) o;
        return (a.getId() == this.getId());
    }

    // toString: Übung
}

```

Sehen wir uns zuerst die Attribute an. Die beiden Attribute `id` und `beginOfContract` machen wir mit `final` wie gefordert zu Konstanten. Dem Attribut `idCounter` geben wir den (willkürlichen) Anfangswert 1000. Ab diesem Wert werden die Angestellten-IDs fortlaufend vergeben. Die Rolle `boss` aus dem UML-Diagramm implementieren wir als Attribut vom Typ `Employee`. Hat ein Angestellter keinen Vorgesetzten, hat `boss` den Wert `null`.

Kommen wir nun zum Konstruktor. Da die ID automatisch vergeben wird, sehen wir dafür keinen Eingabeparameter vor. Wir setzen deren Wert direkt auf den aktuellen Wert des Counters (`this.id = idCounter`), denn wir haben für `id` keine Set-Methode vorgesehen. Da `id` eine Konstante ist, ist nur einmal eine Wertzuweisung möglich. Der Counter wird danach für die nächste Angestelltennummer erhöht (`incrementIdCounter()`). Auch für den Vorgesetzten gibt es keinen Eingabeparameter, denn ein Angestellter muss keinen Vorgesetzten haben. Ein Angestellter wird erst einmal ohne Vorgesetzten erzeugt (`this.boss = null`). Später kann man mit `linkBoss` einen Vorgesetzten zuweisen. Beim Vertragsbeginn sind laut Beschreibung ungültige Werte durch Eingabefehler möglich. Deshalb wird zuerst geprüft, ob ein solcher ungültiger Wert vorliegt. Falls das der Fall ist, wird eine geprüfte Ausnahme geworfen. Der Code dafür befindet sich direkt im Konstruktor, denn wir haben für `beginOfContract` keine Set-Methode vorgesehen. Der Name wird normal über eine Set-Methode initialisiert. Diese kann eine geprüfte Ausnahme werfen. Als Ausnahmeklasse benutzen wir `IllegalArgumentException`. Alle Ausnahmen werden (wie in der ersten Fallstudie) an den Aufrufer weitergereicht. Dadurch können wir später in der grafischen Benutzerschnittstelle mit einer individuellen Fehlerbehandlung reagieren, zum Beispiel mit bestimmten Warndialogen.

Neue Aspekte enthalten die Verwaltungsmethoden zur Vorgesetzten-Beziehung:

1. `linkBoss`:
Wird wie eine Set-Methode implementiert. Bei ungültigen Werten wird eine geprüfte Ausnahme geworfen und weitergereicht.
2. `unlinkBoss`:
Da die Vorgesetzten-Beziehung optional ist (ein Angestellter kann einen Vorgesetzten haben, muss aber nicht), kann `boss` auch wieder auf `null` gesetzt werden. Dafür wird diese Methode verwendet. Im Unterschied zur Alternative `linkBoss(null)` sieht man dieser Methode schon am Namen an, welche Wirkung sie hat. Das macht den Code leichter lesbar.
3. `getlinkBoss`:
Wird grundsätzlich wie eine Get-Methode implementiert. Allerdings benutzen wir als Rückgabotyp `Optional<Employee>` statt `Employee`. Die Klasse `Optional` ist eine Hüllklasse für beliebige Objekte, die einen Wert enthalten kann, der möglicherweise `null` ist. Und genau das ist hier bei uns der Fall. Die Klassenmethode `ofNullable` verpackt einen übergebenen Wert in einen solchen `Optional`-Container. Wird `null` übergeben, wird ein leeres `Optional`-Objekt erzeugt. So zwingt man den Aufrufer der Methode, den Rückgabewert zu überprüfen. Gibt man nur den Wert von `boss` als `Employee`-Objekt zurück, so kann die Überprüfung des Rückgabewerts weggelassen (oder übersehen) werden – und das kann im weiteren Verlauf zu einem Laufzeitfehler führen (`NullPointerException`). Mit der `Optional`-Methode `get` kommt man an den verpackten Wert, und mit `isEmpty` überprüft man, ob das `Optional`-Objekt leer ist oder nicht.
4. `checkBoss`:
Ist eine Objektmethode und überprüft, ob der übergebene Vorgesetzte für dieses Objekt zu einem Zyklus führen würde. Das passiert im einfachsten Fall, falls man das Objekt selbst als seinen eigenen Vorgesetzten übergeben würde. Ist dies nicht der Fall, und hat der übergebene Vorgesetzte selbst keinen direkten Vorgesetzten, liegt kein Zyklus vor. Hat der übergebene

Vorgesetzte selbst einen direkten Vorgesetzten, macht man dieselben Überprüfungen nun rekursiv für diesen Vorgesetzten.

Die `equals`-Methode überschreiben wir so, dass sie `true` liefert, falls beide Objekte Angestellte mit derselben ID sind. Das kann man zum Beispiel nach folgendem Schema machen

1. Wird `null` übergeben, wird `false` geliefert.
2. Wird ein Objekt übergeben, das nicht vom Typ `Employee` ist, wird `false` geliefert. Für die Überprüfung des Typs verwenden wir hier `instanceof`. So kann auch der Vergleich eines `Employee`-Objekts und eines `TemporaryEmployee`-Objekts den Wert `true` liefern. Möchte man, dass nur Objekte ein und derselben Klasse gleich sein können, verwendet man `getClass` statt `instanceof`.
3. Jetzt kann das übergebene Objekt auf den Typ `Employee` gecastet werden (beachte: bisher war es vom Typ `Object`!). Dadurch können wir auf die Get-Methoden von `Employee` zugreifen, um Attributwerte zu vergleichen.
4. Wir vergleichen die IDs. Es wird `true` geliefert, falls diese gleich sind.

Sehen wir uns als nächstes die Klasse `TemporaryEmployee` an:

```
import java.time.LocalDate;

public class TemporaryEmployee extends Employee {

    private LocalDate endOfContract;

    public TemporaryEmployee(String name, LocalDate beginOfContract,
        LocalDate endOfContract)
        throws IllegalArgumentException {
        super(name, beginOfContract);
        setEndOfContract(endOfContract);
    }

    // Methoden für endOfContract: Übung

    // toString: Übung
}
```

Diese Klasse erbt alle Attribute und Methoden der Oberklasse `Employee` und erweitert diese um das zusätzliche Attribut `endOfContract` mit zugehörigen Verwaltungsmethoden.

Die Implementierung des Containers sieht schließlich wie folgt aus:

```
import java.util.ArrayList;
import java.util.Iterator;

public class EmployeeContainer implements Iterable<Employee> {

    private ArrayList<Employee> allEmployees;

    public EmployeeContainer() {
```

```

        allEmployees = new ArrayList<Employee>();
    }

    public void addEmployee(Employee a) {
        if (allEmployees.contains(a))
            throw new IllegalArgumentException("Employee already
                exists");
        allEmployees.add(a);
    }

    public void removeEmployee(Employee a) {
        if (!allEmployees.contains(a))
            return;
        for (Employee b : allEmployees) {
            if (b.getlinkBoss().isEmpty())
                continue;
            if (a.equals(b.getlinkBoss().get()))
                b.unlinkBoss();
        }
        allEmployees.remove(a);
    }

    public Iterator<Employee> iterator() {
        return allEmployees.iterator();
    }
}

```

In der `addEmployee`-Methode überprüfen wir mit `contains`, ob der Angestellte, den wir in den Container aufnehmen wollen, schon im Container enthalten ist. Da in `contains` dafür `equals` benutzt wird, bedeutet das: `contains` liefert `true`, falls es schon einen Angestellten mit derselben ID gibt. Da IDs automatisch vergeben werden, deutet das auf einen Programmierfehler hin. Deshalb werfen wir hier eine ungeprüfte API-Ausnahme (und reichen diese weiter).

In der `removeEmployee`-Methode wollen wir einen Angestellten aus dem Container entfernen. Nun kann dieser Angestellte aber Vorgesetzter anderer Angestellter sein. Dort müssen wir ihn zuerst als Vorgesetzten austragen, da sonst ein inkonsistenter Datenzustand entsteht (keine Referenzen mehr auf den Angestellten im Container, aber Referenzen auf den Angestellten als Vorgesetzter von anderen Angestellten). Dazu durchlaufen wir alle Angestellten und überprüfen deren Vorgesetzte.

An dieser zweiten Fallstudie sehen wir damit, dass die Implementierung der Containermethoden auch aufwendiger sein kann, und es unter Umständen nicht ausreicht, einfach Objekte an Methoden einer Collection durchzureichen (wie in der ersten Fallstudie).

Die Implementierung der Klasse `IllegalArgumentException` überlassen wir zur Übung.

Übung: Implementieren Sie spätestens jetzt das komplette Modell mit allen überlassenen Lücken und testen Sie dieses in einem Hauptprogramm.

7.4 Allgemeine Regeln der Implementierung

Jetzt fassen wir noch einmal die wesentlichen Erkenntnisse aus den beiden Fallstudien für die Java-Implementierung von Datenklassen kompakt zusammen.

7.4.1 Aufbau einer Klasse

Technische Details: Unterkapitel 4.2.1

UML-Klassen werden in Java ebenfalls als Klassen implementiert. Ein Java-Klasse ist wie folgt aufgebaut:

```
<Klassenkopf> {  
    <Attributliste>  
    <Konstruktorliste>  
    <Methodenliste>  
}
```

Jede Java-Klasse definiert einen Datentyp. Dabei sind die Attribute die Komponenten des Datentyps und die Methoden dienen der Verwaltung der in den Attributen abgelegten Daten (zum Beispiel zum Lesen, Schreiben, zur Manipulation oder Berechnung von Daten). Mithilfe der Konstruktoren lassen sich Objekte der Klasse erzeugen und initialisieren.

7.4.2 Attribute einer Klasse

Technische Details: Unterkapitel 4.2.2

Objekt- und Klassenattribute von UML-Klassen werden als Klassen- und Objektattribute in Java implementiert. Rollen in Objektbeziehungen in UML werden als Objektattribute in Java implementiert. Java-Attribute haben üblicherweise die Sichtbarkeit `private` und können nur über öffentliche Verwaltungsmethoden gelesen und geschrieben werden. Damit lassen sich vorhandene Datenstrukturinvarianten sicherstellen. Für deren Implementierung unterscheiden wir zwischen folgenden Arten von Attributen:

- **Einwertige Attribute**

Das sind Attribute, die genau einen Wert haben können. Für diese wählen wir einen möglichst gut zur Anwendungsbeschreibung passenden Datentyp. Dafür kommen primitive Datentypen (wie `int`), API-Klassen (wie `String`) oder selbst definierte Datentypen (wie `Point`) in Frage.

- **Mehrwertige Attribute**

Das sind Attribute, die mehrere Werte haben können. Für diese wählen wir eine passend parametrisierte API-Sammlung als Datentyp. Den Parameter wählen wir dabei geeignet für einen Wert der Sammlung. Handelt es sich um eine Sammlung primitiver Werte, verwenden wir eine passende Hüllklasse als Parameter.

- **Kann-Attribute**

Das sind Referenz-Attribute, für die es Objekte gibt, die für dieses Attribut keinen Wert haben. Für solche Objekte geben wir dem Attribut den Wert null. Diesen Wert benutzen wir standardmäßig als Anfangswert für jedes neu erstellte Objekt.

- **Muss-Attribute**

Das sind Referenz-Attribute, für die alle Objekte einen Wert haben müssen. Ist das Attribut mehrwertig, bedeutet das, dass die Liste der Werte nicht leer sein darf.

7.4.3 Konstruktoren einer Klasse

Technische Details: Unterkapitel 4.2.3

Für eine Klasse muss man nicht zwingend explizit einen Konstruktor angeben. Gibt man keinen Konstruktor an, so wird automatisch ein leerer Konstruktor ergänzt. Das ist ein Konstruktor, bei dem die Parameterliste leer ist, und der keinerlei Initialisierungen vornimmt (die Werte der Attribute bleiben also auf den Standardwerten). Definiert man aber explizit einen Konstruktor, so hat die Klasse nicht automatisch auch einen leeren Konstruktor. Dieser muss, wenn gewünscht, dann auch explizit hinzugefügt werden.

Konstruktoren initialisieren die Attribute eines neu angelegten Objekts. Man ergänzt Konstruktoren passend zu Beschreibung der Anwendung. Neue Attributwerte werden als Eingabeparameter übergeben. Üblicherweise heißen die Eingabeparameter dabei genauso wie die Attribute.

Für Kann-Attribute sieht man üblicherweise keine Eingabeparameter vor. Einwertige Kann-Attribute initialisiert man standardmäßig mit dem Wert null, und mehrwertige Kann-Attribute mit einer leeren Liste.

Für Muss-Attribute sieht man entweder einen Eingabeparameter vor, oder man setzt in der Attribut-Deklaration oder im Konstruktor einen festen Anfangswert. Im Konstruktor verwendet man grundsätzlich Set- und Link-Methoden zum Initialisieren von Attributen, um redundanten Code zu vermeiden. Dabei auftretende Ausnahmen werden weitergereicht. Eine Ausnahme bilden Konstanten, zu denen es ja keine Set-Methoden gibt.

Es kann mehrere Konstruktoren mit unterschiedlichen Parameterlisten für unterschiedliche Anwendungszwecke geben. Insbesondere wird oft ein sogenannter Kopierkonstruktor ergänzt, mit dem man eine Kopie gleichen Inhalts eines übergebenen Objekts anlegen kann.

Eigenschaft eines Attributs	Initialisierung im Konstruktor
Mehrwertiges kann-Attribut	ja, als leere Liste <i>Alternativ:</i> Leere Liste als Anfangswert in Deklaration
Einwertiges Kann-Attribut	nein <i>Stattdessen:</i> null als Anfangswert in Deklaration
Muss-Attribut	ja, über Eingabeparameter <i>Alternativ:</i> fester Standardwert im Konstruktor

Tabelle 1: Initialisierung von Attributen in Konstruktoren

7.4.4 Verwaltung von Attributen

Technische Details: Unterkapitel 4.2.4

Klassen- und Objektmethoden von UML-Klassen werden als Klassen- und Objektmethoden in Java implementiert. Java-Methoden haben üblicherweise die Sichtbarkeit `public`, damit wir in anderen Klassen auf die Attributwerte zugreifen können. Verschiedene Standardverwaltungsmethoden sind:

1. Get- und Getlink-Methoden

Get-Methoden liefern den Wert von Attributen und Getlink-Methoden den Wert von Rollen aus dem UML-Modell. Bei der Implementierung unterscheiden wir folgende Fälle:

- **Mehrwertige Attribute:**

Kann ein Attribut mehrere Werte haben, so geben wir die komplette Liste aller Werte zurück (zum Beispiel als Array oder als Sammlung). Diese Liste darf auch leer sein. Wollen wir die Implementierung vom Datentyp des Attributs entkoppeln, implementieren wir alternativ zu einer Get- oder Getlink-Methode die Schnittstelle `Iterable`, um auf die Liste aller Werte zuzugreifen.

- **Einwertige Kann-Attribute:**

Ein einwertiges Attribut ist optional oder ein Kann-Attribut, wenn es Objekte gibt, die für dieses Attribut keinen Wert haben. Für solche Objekte geben wir dem Attribut in der Regel den Wert `null` (beachten Sie, dass das nur für Referenz-Datentypen möglich ist!). Als Rückgabetyt verwenden wir für Kann-Attribute ein `Optional`-Objekt, in das wir den Attributwert verpacken. Die Klasse `Optional` ist eine Hüllklasse, die ein beliebiges Objekt oder `null` enthalten kann. Die Klassenmethode `ofNullable` verpackt den übergebenen Wert in einen solchen `Optional`-Container. Wird `null` übergeben, wird ein leeres `Optional`-Objekt erzeugt. So zwingt man den Aufrufer der Methode, den Rückgabewert zu überprüfen. Mit der `Optional`-Methode `get` kommt man an den verpackten Wert, und mit `isEmpty` überprüft man, ob das `Optional`-Objekt leer ist oder nicht.

- **Einwertige Muss-Attribute:**

Für einwertige nicht-optionale Attribute oder Muss-Attribute verwenden wir als Rückgabetyt den Datentyp des Attributs.

- **Attribute mit unveränderlichem Datentyp:**

Hat ein Attribut einen Referenz-Datentyp, dessen Objekte unveränderbar sind (wie zum Beispiel `String` oder `LocalDate`), liefern wir den originalen Attributwert zurück (denn es kann zu keinen Nebeneffekten kommen).

- **Attribute mit veränderlichem Datentyp:**

Hat ein Attribut einen Referenz-Datentyp, dessen Objekte veränderbar sind, sollte nur eine Kopie des Attributwerts geliefert werden. Ansonsten könnte man über die gelieferte Referenz auf den Originalwert zugreifen, diesen verändern und so Datenstrukturinvarianten verletzen.

2. Check-Methoden

Check-Methoden dienen der Überprüfung von Datenstrukturinvarianten von Attributwerten. Diese legt man oft nur als private interne Hilfsmethoden an und stellt sie nicht nach

Eigenschaft des Attributs	Implementierung
Mehrwertiges Attribut	Get- und Getlink-Methode gibt Liste aller Attributwerte zurück <i>Alternative:</i> Implementiere Schnittstelle Iterable
Einwertiges Kann-Attribut	Get- und Getlink-Methode gibt Optional-Objekt mit dem Attributwert als Inhalt zurück
Einwertiges Muss-Attribut	Get- und Getlink-Methode gibt Attributwert direkt zurück
Attribut hat veränderlichen Datentyp	Get- und Getlink-Methode gibt Kopie des Attributwerts zurück
Attribut hat unveränderlichen Datentyp	Get- und Getlink-Methode gibt originalen Attributwert zurück

Tabelle 2: Implementierung von Get- und Getlink-Methoden

außen zur Verfügung. Je nachdem, ob sich die Überprüfung abhängig von Attributwerten des Objekts ist oder nicht, handelt es sich um eine Klassen- oder Objektmethode.

3. Set-, Add-, Remove-, Link- und Unlink-Methoden

Set-, Add- und Remove-Methoden verändern den Wert von Attributen und Link- und Unlink-Methoden den Wert von Rollen aus dem UML-Modell. Gelten für ein Attribut Datenstrukturinvarianten, so überprüft man diese mit Check-Methoden. Soll ein ungültiger Wert geschrieben werden, wirft man eine Ausnahme und reicht diese weiter. Bei der Implementierung unterscheiden wir folgende Fälle:

- **Mehrwertige Attribute**

Bei mehrwertigen Attributen verwenden wir eine Add-Methode, um einen Attributwert hinzuzufügen, und eine Remove-Methode, um einen Attributwert zu entfernen.

- **Einwertige Attribute**

Bei einwertigen Attributen verwenden wir stattdessen eine Set-Methode (für Attribute aus dem UML-Modell) beziehungsweise eine Link-Methode (für Rollen aus dem UML-Modell), um den Attributwert zu überschreiben. Handelt es sich um ein Kann-Attribut, verwenden wir zudem eine Unlink-Methode, um den Attributwert auf `null` zu setzen.

- **Attributwerte mit freier Benutzereingabe**

Ist eine freie Benutzereingabe für einen Attributwert möglich, werfen wir eine geprüfte Ausnahme für ungültige Attributwerte. Dadurch muss diese vom Aufrufer behandelt werden.

- **Attributwerte ohne freie Benutzereingabe**

Ist keine freie Benutzereingabe für einen Attributwert möglich, werfen wir eine ungeprüfte Ausnahme für ungültige Attributwerte. Dadurch muss diese nicht vom Aufrufer behandelt werden. In diesem Fall deuten auftretende Ausnahmen auf Programmierfehler hin.

Eigenschaft des Attributs	Implementierung
Mehrwertiges Attribut	Add-Methode fügt Wert hinzu Remove-Methode entfernt Wert
Einwertiges Kann-Attribut	Set- und Link-Methode überschreibt Wert Unlink-Methode setzt Wert auf null
Einwertiges Muss-Attribut	Set- und Link-Methode überschreibt Wert Hat keine Unlink-Methode
Freie Benutzereingabe für Attributwert möglich	Wirft geprüfte Ausnahme bei ungültigen Werten
Keine freie Benutzereingabe für Attributwert möglich	Wirft ungeprüfte Ausnahme bei ungültigen Werten

Tabelle 3: Implementierung von Set-, Add-, Remove-, Link- und Unlink-Methoden

7.4.5 equals überschreiben

Nach der Standardimplementierung von `equals` in der Klasse `Object` sind zwei Objekte gleich, wenn sie identisch sind, also an der gleichen Speicheradresse stehen. Diese Implementierung ist für manche Klassen sinnvoll, für andere nicht. Für Datenklassen ist es oft praktisch, zwei Objekte als gleich zu betrachten, wenn alle oder bestimmte ihrer Attributwerte übereinstimmen. Um solche Änderungen in Bezug auf die Gleichheit von Objekten zu implementieren, muss eine Klasse die `equals`-Methode überschreiben.

Wichtig hierbei ist insbesondere zu berücksichtigen, dass `equals` implizit in anderen API-Klassen benutzt wird. So wird `equals` in der Collection-Methode `contains` verwendet, um zu überprüfen, ob ein Objekt schon in einer Collection enthalten ist.

Da man die Datenobjekte einer Anwendung standardmäßig in Collections verwaltet, sollte `equals` passend zur erwünschten Objektverwaltung implementiert werden. In manchen Anwendungen ist es erlaubt, dass zwei verschiedene Objekte den gleichen Inhalt haben. Im grafischen Editor dürfen zum Beispiel zwei Kreise durchaus deckungsgleich übereinander liegen. In diesem Fall muss man `equals` nicht überschreiben.

In anderen Anwendungen ist es nicht erlaubt, dass zwei verschiedene Objekte den gleichen Inhalt haben. Zum Beispiel sollte es nicht zwei unterschiedliche Angestellte mit derselben ID geben. In diesem Fall muss man `equals` passend überschreiben.

Eine Implementierung von `equals` muss dabei folgende Eigenschaften haben, wobei `x`, `y`, `z` keine `null`-Referenzen sind:

- **Reflexivität:** Ein Objekt `x` soll gleich zu sich selbst sein. Rufen wir also `x.equals(x)` auf, so soll der Wahrheitswert `true` zurückgegeben werden.
- **Symmetrie:** Für zwei Objekte `x` und `y` soll es egal sein, von welchem man die `equals`-Methode aufruft und mit dem anderen vergleicht. Die Aufrufe `x.equals(y)` und `y.equals(x)` sollen also immer dasselbe Ergebnis liefern.

- **Transitivität:** Sind die Objekte `x` und `y` nach Aufruf der `equals`-Methode gleich und die Objekte `y` und `z` ebenfalls, so soll `equals` auch für `x` und `z` den Wahrheitswert `true` zurückgeben. Anders gesagt: Wenn `x.equals(y) == true` und `y.equals(z) == true`, dann muss auch `x.equals(z) == true` gelten.
- **Konsistenz:** Der Aufruf `x.equals(y)` liefert immer das gleiche Ergebnis, sofern `x` und `y` sich bei den Aufrufen nicht unterscheiden. Die Methode `equals` verhält sich also unabhängig davon, wo und wann sie aufgerufen wird, immer gleich.
- **Verhalten bei null-Referenzen:** Wird an die Methode `equals` als Parameter `null` übergeben, soll das Ergebnis immer `false` sein – egal, wie `x` aussieht. Der Aufruf `x.equals(null)` soll also immer `false` zurückgeben.

Die ersten drei genannten Eigenschaften bedeuten, dass `equals` eine sogenannte **Äquivalenzrelation** ist. Solche haben sie bereits in der Veranstaltung *Diskrete Strukturen und Logik* kennengelernt.

Als Daumenregel, wie eine `equals`-Methode aufgebaut sein sollte, können Sie das folgende Schema benutzen:

Merke 7.3: Schema zur Implementierung einer `equals`-Methode in einer Klasse `K`

```
@Override
public boolean equals(Object other) {
    if(other == null)
        return false;
    if(!(other.getClass().equals(this.getClass()))
    )
        return false;
    K another = (K) other;
    return <VergleichAttributwerte>;
}
```

1. Prüfung des übergebenen Objekts auf `null`: Wird `null` übergeben, wird `false` geliefert.
2. Überprüfung der Klassenzugehörigkeit des übergebenen Objekts mit `getClass`: Sind die Klassen unterschiedlich, wird `false` geliefert. Manchmal will man in einer Vererbungshierarchie Objekte aus Unter- und Oberklassen als gleich ansehen. In diesem Fall verwendet man `instanceof` statt `getClass`.
3. Typcast des zu übergebenen Objekts auf die Klasse, in der geprüft wird (das ist nach der vorherigen Überprüfung möglich). Da der Typ des Eingabeparameters `Object` ist, können wir sonst im nächsten Schritt für das übergebene Objekt nicht auf die Methoden der prüfenden Klasse zugreifen können.
4. Prüfung auf Gleichheit ausgewählter Attributwerte.

In Entwicklungsumgebungen wie Eclipse kann man sich `equals` auch automatisch generieren lassen. Zur besseren Übung und Klausurvorbereitung sollten wir die Methode aber in *Informatik 2* stets selbst implementieren.

7.4.6 hashCode überschreiben

Die Methode `hashCode` generiert zu einem Objekt einen sogenannten **HashCode**. Das ist eine Art Prüfsumme, deren Details Sie in der Veranstaltung *Informatik 3* kennenlernen werden. In *Informatik 2* sind Hashcodes ausschließlich für diesen Abschnitt relevant. Hashcodes werden insbesondere für die Verwendung von Hash-basierten Collections (zum Beispiel `HashMap` und `HashSet`) benötigt. Die Implementierung von `hashCode` soll insofern konsistent mit der `equals`-Implementierung sein, dass zwei Objekte `x` und `y` denselben Hashcode erhalten, wenn sie nach `equals` als gleich gelten:

$$(x.equals(y) == true) \implies (x.hashCode() == y.hashCode()).$$

Generell sollte jede Klasse, die `equals` überschreibt, auch `hashCode` überschreiben, um deren Konsistenz mit `equals` zu gewährleisten. Mit der `hash`-Methode der Utility-Klasse `Objects` lassen sich Hashcodes berechnen. In dieser Vorlesung werden wir dieses Thema allerdings nicht weiter behandeln. Wie bei `equals` kann man sich auch eine Implementierung für `hashCode` automatisch in Entwicklungsumgebungen wie Eclipse generieren lassen.

7.4.7 Containerklassen zu Datenklassen

Zur Laufzeit erstellte Objekte einer Datenklasse verwaltet man in einer zugehörigen Containerklasse. Intern speichert man die Objekte in einer privaten Sammlung. Der Container kapselt die interne Sammlung und stellt nur spezifische eingeschränkte Zugriffsmöglichkeiten zur Verfügung. Insbesondere kann man standardmäßig neue Objekte hinzufügen (Add-Methode) und Objekte löschen (Remove-Methode). Zudem macht man den Container über das **Iteratormuster** einheitlich durchlaufbar. Will man ein Objekt hinzufügen, muss man gegebenenfalls beachten, ob es schon dasselbe Objekt im Container gibt und ob in der Anwendung Dubletten erlaubt sind. Löscht man ein Objekt aus dem Container, muss man darauf achten, vorher alle gegebenenfalls vorhandenen Beziehungen des Objekts zu anderen Objekten abzubauen.

7.5 Einführung in Javadoc

Das Erstellen und Pflegen einer externen Dokumentationsdatei zusätzlich zum Quellcode ist sehr nützlich für Personen, die sich in das Programm oder den bestehenden Code einarbeiten wollen, wird aber oft als lästig empfunden und daher bei der Softwareentwicklung häufig vernachlässigt. Um diesen Effekt zu umgehen, kann man im Quellcode spezielle **Dokumentationskommentare** einfügen, aus denen automatisch eine Dokumentation generiert werden kann. Das zugehörige Tool

nennt sich **Javadoc**. Es erzeugt automatisch eine Dokumentation im HTML-Format. Damit Kommentare von Javadoc erkannt werden, müssen sie in der folgenden Form im Quellcode hinterlegt sein:

```
/**
 * Mehrzeiliger Kommentar
 */
```

Dabei ist es wichtig, dass der Kommentar mit zwei Sternen beginnt (**/****), damit der Kommentar vom Javadoc-Generierungstool erkannt wird. Solche Kommentare sind insbesondere an folgenden Stellen einzufügen:

1. **Zur Beschreibung einer Klasse:** Direkt vor der Klassendefinition
2. **Zur Beschreibung eines Attributs:** Direkt vor der Attributsdeklaration
3. **Zur Beschreibung eines Konstruktors oder einer Methode:** Direkt vor der Konstruktor- oder Methodendefinition. Der erste Absatz des Kommentars ist dabei üblicherweise eine Schilderung des Konstruktors oder der Methode. Danach können verschiedene Schlüsselworte angegeben werden, die im folgenden Absatz näher erklärt werden.

Javadoc versteht die folgenden Schlüsselworte und baut zugehörige Beschreibungen übersichtlich in die Dokumentation ein:

1. **@param:**
Wird für jeden Eingabeparameter einer Methode oder eines Konstruktors verwendet.
Verwendung: **@param** <Variable> <Beschreibung>
2. **@return:**
Zur Beschreibung des Rückgabewerts einer Methode.
Verwendung: **@return** <Beschreibung>
3. **@throws:**
Zur Beschreibung einer Ausnahme (geprüft oder ungeprüft) einer Methode oder eines Konstruktors.
Verwendung: **@throws** <Ausnahme> <Beschreibung>

Hier Beispiele aus der zweiten Fallstudie:

```
/**
 * Repräsentiert einen Mitarbeiter mit eindeutiger ID,
 * Namen, Vertragsbeginn und optionalem Vorgesetzten.
 *
 * Jeder Mitarbeiter erhält automatisch eine eindeutige ID,
 * die auf einem internen Zähler basiert.
 *
 * Folgende Bedingungen gelten:
 * - Der Name muss mindestens zwei Zeichen lang sein.
 * - Der erste Buchstabe des Namens muss groß sein.
 * - Der Vertragsbeginn muss nach dem 01.01.2000 liegen.
 * - Beziehungen zu Vorgesetzten dürfen keine Zyklen bilden.
 *
 * @author Robert Lorenz
```

```

    * @version 1.0
    */
    public class Employee {

        /**
         * Name des Mitarbeiters.
         */
        private String name;
        ...

        /**
         * Erzeugt einen neuen Mitarbeiter ohne Vorgesetzten.
         * Die ID wird automatisch vergeben.
         * @param name Name des Mitarbeiters
         * @param beginOfContract Beginn des Arbeitsvertrags
         * @throws IllegalArgumentException falls Name oder Vertragsbeginn
         *         ungültig sind
         */
        public Employee(String name, LocalDate beginOfContract) throws
            IllegalArgumentException {
            ...
        }

        /**
         * Prüft, ob ein Name gültig ist.
         * Ein gültiger Name:
         * - besitzt mindestens zwei Zeichen
         * - beginnt mit einem Großbuchstaben
         * @param name zu prüfender Name
         * @return true, wenn der Name gültig ist, sonst false
         */
        private static boolean checkName(String name) {
            ...
        }

        /**
         * Setzt den Namen des Mitarbeiters.
         * @param name neuer Name
         * @throws IllegalArgumentException falls der Name ungültig ist
         */
        public void setName(String name) throws IllegalArgumentException {
            ...
        }

        /**
         * Liefert den Namen des Mitarbeiters.
         */
    }

```

```

    * @return Name des Mitarbeiters
    */
    public String getName() {
        ...
    }
    ...
}

```

Die HTML-Dokumentation, die Javadoc erstellt, sieht genauso aus wie die Original Oracle Java Dokumentation. So lassen sich ganz einfach eigene Klassen um Javadoc-Kommentare erweitern und eine Javadoc-Dokumentation erstellen, indem man auf der Kommandozeile `javadoc` gefolgt von allen `.java`-Dateien aufruft, die in die Dokumentation aufgenommen werden sollen.

7.6 Annotationen

Kommentare und Javadoc richten sich ausschließlich an Menschen, doch sind diese nicht die einzigen, für die Kommentare und Hinweise zum Code interessant sein können. Manchmal wollen wir auch dem Compiler etwas mitteilen, wofür es sogenannte **Annotationen** gibt. Diese können den Programmcode mit verschiedenen Zusatzinformationen versehen, die vom Compiler geprüft werden. Annotationen haben die Form

```
@<Annotation>
```

und sind optional. Das Paket `java.lang` stellt einige nützliche Annotationen zur Verfügung, darunter die Folgenden:

1. `@Override`:
 - **Ort**: Vor einer Methodendefinition.
 - **Bedeutung**: Zeigt an, dass diese Methode eine Methode der Oberklasse überschreibt.
 - **Wirkung**: Compiler überprüft die verwendete Methode (muss genau so in der Oberklasse vorhanden sein) und meldet gegebenenfalls einen Fehler.
2. `@Deprecated`:
 - **Ort**: Vor Definition eines API-Elements (Klasse, Schnittstelle, Methode, Konstruktor, Attribut).
 - **Bedeutung**: Zeigt an, dass dieses Element veraltet ist und nicht mehr verwendet werden sollte (es wird in einer kommenden Version entfernt).
 - **Wirkung**: Compiler zeigt eine Warnung an, wenn das Element trotzdem verwendet wird.
3. `@FunctionalInterface`:
 - **Ort**: Vor einer Schnittstellendefinition.
 - **Bedeutung**: Markiert die Schnittstelle als funktional; diese besitzt also nur eine abstrakte Methode, was auch in künftigen Weiterentwicklungen nicht geändert werden soll.

- **Wirkung:** Compiler überprüft die Anzahl der abstrakten Methoden der Schnittstelle und meldet gegebenenfalls einen Fehler.

4. `@SuppressWarnings`:

Form: `@SuppressWarnings(<Einschraenkung>)`

- **Ort:** Vor Definition eines Attributs, einer Methode oder einer Klasse.
- **Bedeutung:** Zeigt an, dass bestimmte Compilerwarnungen für dieses Element unterdrückt werden sollen (zum Beispiel während einer Code-Umstellung oder falls der Entwickler bewusst ein bestimmtes Konzept nutzt, das normalerweise eine Warnung auslöst).
- **Wirkung:** Compiler unterdrückt die angegebenen Warnungen.
- **Einsatz:** `<Einschraenkung>` gibt an, welche Warnungen unterdrückt werden; mögliche Werte sind zum Beispiel "all", "serial", "unchecked", "rawtypes", "resource", "deprecation" (Sie sind eingeladen, durch Eigenrecherche selbst herauszufinden, welche Warnungen damit jeweils unterdrückt werden).

Ein häufiger Fehler ist es, nach Annotationen einen Strichpunkt zu setzen. Dort darf aber **kein** Strichpunkt stehen! Annotationen können auch zur Laufzeit abgefragt werden. Dieses in Zusammenhang mit *Reflection* stehende Konzept wird im Rahmen der Veranstaltung *Informatik 2* nicht weiter behandelt.